

Rapport intermédiaire du projet d'informatique Individuel

Neural Scrum

Prudhommeaux Erwan

Groupe 3



Sommaire

Sommaire.....	2
Contextualisation.....	3
Choix techniques faits.....	3
Architecture de l'application.....	4
Gestion de projet.....	7
Planning prévisionnel.....	7
Planning intermédiaire.....	7
Fonctionnements.....	8
A - Partie modélisation.....	8
B - Partie physique.....	10
Résultats.....	12
Piste d'évolutions & bilan personnel.....	13
Comment utiliser mon code.....	13

Contextualisation

Mon projet porte sur le thème du rugby. Il consiste à modéliser des joueurs répartis en deux équipes adverses, avec un ballon confié à l'un d'eux, et à les faire évoluer au sein d'une simulation informatique basée sur les règles de ce sport. Les décisions prises par les joueurs seront entièrement régies par un modèle d'apprentissage. Cela permettra de voir émerger des stratégies adaptées à des situations de jeu bien définies.

Mon objectif est d'entraîner une IA afin de pouvoir, par la suite, la confronter à différentes situations. En fonction de son apprentissage, j'obtiendrai ainsi une simulation de la meilleure issue possible sur le terrain.

L'idée de ce projet m'est venue pour plusieurs raisons. Passionné par le rugby depuis mon plus jeune âge, je souhaitais le comprendre encore mieux et observer si une IA serait capable de reproduire ce que l'on fait réellement sur le terrain. Toutefois, la véritable origine de ce projet remonte à ma classe préparatoire : j'y avais commencé à modéliser informatiquement toutes les interactions du rugby dans le but de créer un jeu. J'ai donc repris cette idée initiale en y intégrant la dimension d'intelligence artificielle pour concevoir mon projet actuel.

Choix techniques faits

J'ai décidé de développer ce projet en langage Python, d'une part parce que ma modélisation initiale avait été réalisée avec ce langage, et d'autre part car je maîtrisais mieux la création d'IA dans cet environnement.

Concernant la physique du rugby, j'ai commencé par reprendre mon travail précédent, mais j'ai rapidement rencontré un problème : les temps de calcul étaient beaucoup trop longs en raison d'équations trop poussées. J'ai donc décidé de tout simplifier. Pour cela, j'ai modifié le système de passes afin qu'elles s'effectuent directement d'un joueur à un autre, plutôt que vers des zones libres. Quant aux trajectoires, j'ai ajouté un comportement incitant les attaquants à se diriger vers la ligne pour marquer un essai, et les défenseurs (l'équipe qui n'a pas la possession) à se diriger vers le ballon. J'ai ensuite intégré

un modèle d'apprentissage, qui est du renforcement learning, basé sur Micrograd étudié en cours, ce qui m'a beaucoup aidé.

Concernant l'architecture du code et la présentation des résultats, j'ai dû faire des choix décisifs. Pour le code, j'ai opté pour la programmation orientée objet (POO) en utilisant des classes, ce qui m'a permis d'obtenir une structure beaucoup plus claire et propre, tout en optimisant la vitesse d'exécution. Enfin, pour la simulation et le rendu visuel, j'ai choisi de me limiter à quelques joueurs et à de courtes séquences de match. En effet, intégrer trop d'agents augmentait excessivement le temps de réflexion de l'IA. De plus, ce format restreint me permet de configurer des situations de jeu précises et d'analyser beaucoup plus finement les réactions des joueurs.

Architecture de l'application

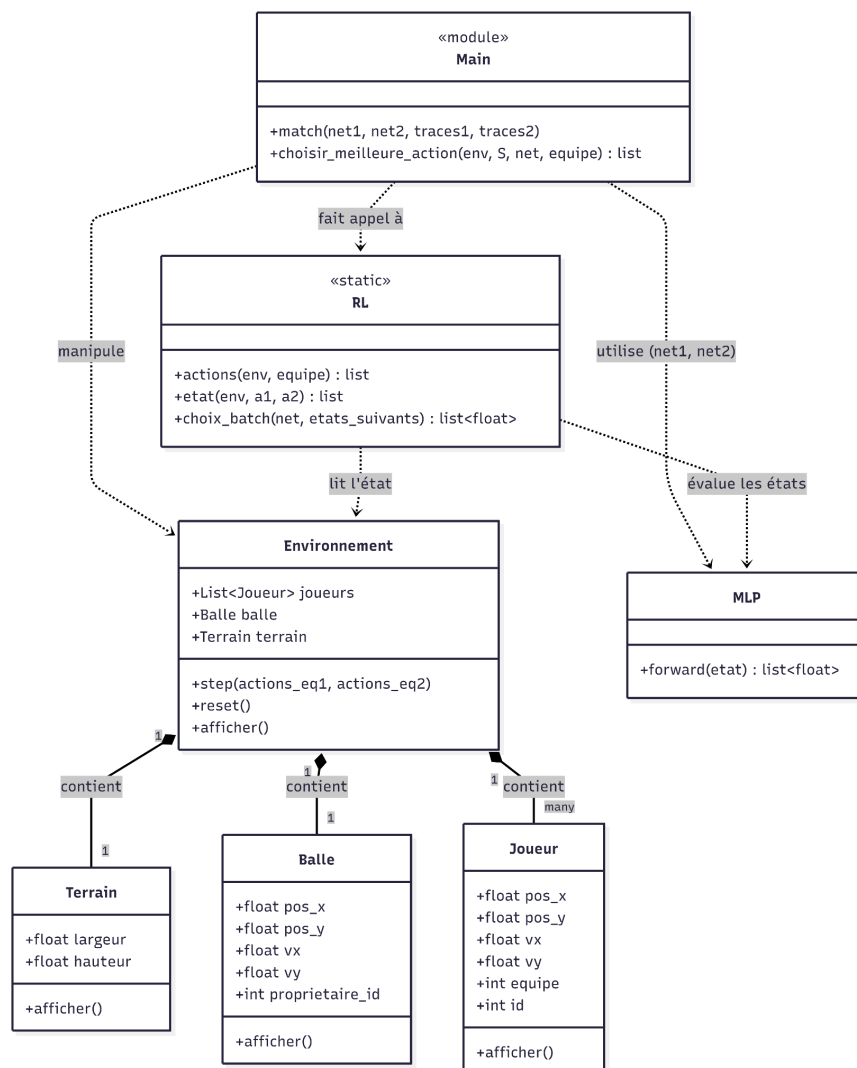


diagramme de classes complet

Voici la représentation en diagramme de classes de mon programme. On y voit mon environnement composé de mon terrain, des joueurs et du ballon qui sont gérés par mon modèle MLP. Le système repose sur un Processus de Décision de Markov qui fait interagir deux entités principales : l'environnement (le simulateur physique) et les agents (les réseaux de neurones).

À chaque instant t , la boucle de simulation fonctionne ainsi :

1. **Observation** : Le système extrait l'état actuel de l'environnement (positions, vitesses, possession de balle).

2. Décision : Les réseaux de neurones (net1,net2) évaluent les actions possibles et choisissent la meilleure.
3. Action : Les actions sont envoyées à l'environnement via la fonction env.step().
4. Transition : L'environnement calcule la physique et génère l'état t+1

Mon code repose sur plusieurs principes de fonctionnement répartis en différents scripts. La partie liée au modèle se trouve dans les fichiers `engine.py`, `nn.py` et `fonction_RL.py`, tandis que toute la modélisation physique du rugby est regroupée dans `document.py`. Le fichier `main.py` me permet d'initialiser et de lancer l'apprentissage des réseaux de neurones, tout en y configurant les paramètres souhaités. Enfin, pour évaluer les résultats de cet apprentissage, j'utilise un script de test dédié : `test_ia.py`.

Le fichier `fonction_RL.py` rassemble l'ensemble des fonctions nécessaires à l'apprentissage de mon modèle MLP, dont l'architecture est définie dans le fichier `nn.py`.

Quant à l'affichage, il est géré dans le script principal (`main.py`), où j'instancie les différents objets pour les représenter visuellement selon mes besoins. Les deux équipes sont distinguées par deux couleurs : l'une bleue et l'autre rouge. Le ballon est modélisé en jaune et s'affiche sur le joueur qui le possède. Lors de la simulation, j'affiche consécutivement 60 étapes d'affilée, chacune correspondant à un pas de temps de 1 seconde.

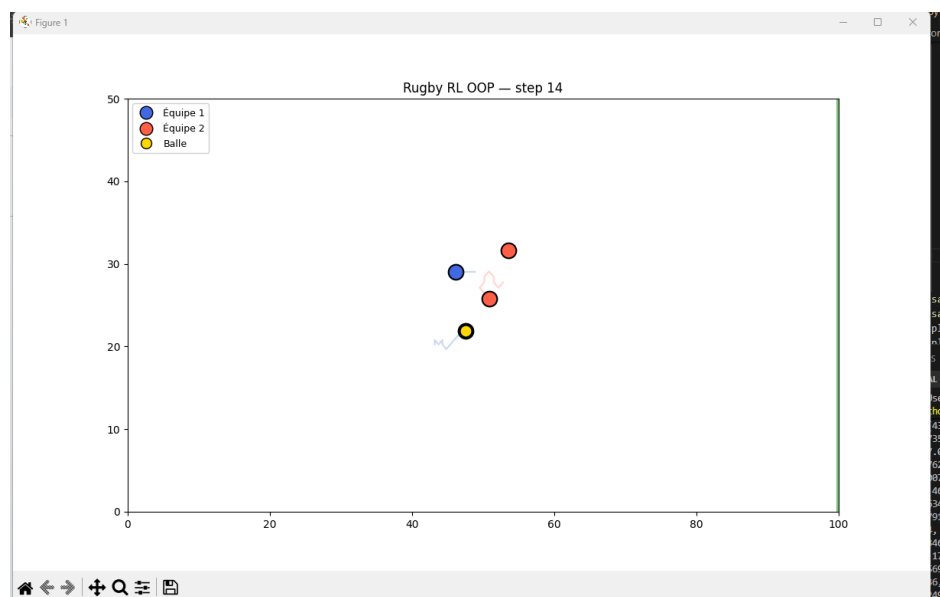


Image d'un step lors de l'affichage d'une époque

Gestion de projet

Planning prévisionnel

[illegible]

Planning intermédiaire

	Janv	Février	Mars	Avril
Prise en main				
Test et amélioration de type de neurones				
Application des neurones au projet				
Transition en POO				
Simplification de la simulation du rugby				
Préparation de la soutenance				

J'ai dû adapter mon planning suite à la réévaluation du temps nécessaire à la création du réseau de neurones. Lors d'une discussion avec mon tuteur, j'ai compris que la programmation orientée objet (POO) apporterait une réelle valeur ajoutée à mon projet. J'ai ensuite suivi un cours sur les modèles MLP en programmation objet, ce qui m'a permis de restructurer mon code. J'ai donc révisé mon calendrier initial pour y intégrer ces nouvelles étapes de développement.

Fonctionnements

A - Partie modélisation

Mon code utilise un espace d'actions hybride, combinant un mouvement continu discrétisé et un choix de cible. Chaque action est modélisée sous la forme d'un vecteur : $[F_x, F_y, \text{cible}]$.

- Composante Motrice (F_x, F_y) : Les mouvements sont définis par des forces appliquées aux joueurs. Pour permettre au réseau de neurones d'explorer efficacement sans être submergé, cet espace continu a été discrétisé. Les forces prennent des valeurs dans un ensemble fini (ex: -50, 0, 50), offrant 8 directions de déplacement plus l'immobilité.
- Composante Interactive (cible) : Représentée par un entier (l'identifiant du joueur cible, ou -1 s'il n'y a pas de passe).

Pour évaluer la qualité d'une action, le système utilise un Perceptron Multicouche (MLP - Multi-Layer Perceptron).

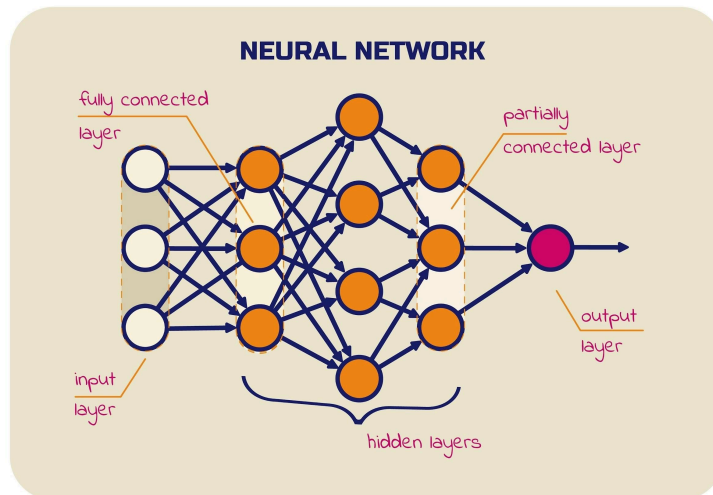


image générée par IA d'un réseaux de neurone

Dans l'apprentissage par renforcement classique (Q-Learning), on utilise généralement une table pour stocker la valeur de chaque paire état-action. Cependant, avec des coordonnées spatiales continues (x, y), le nombre d'états possibles tend vers l'infini, rendant cette approche impossible. Le Perceptron Multicouche (MLP) résout ce problème en agissant comme un approximateur de fonction.

Plutôt que d'évaluer une paire État-Action $Q(s, a)$, mon code simule l'issue d'une action pour générer un état futur potentiel (appelé afterstate ou s_{t+1}). Le réseau évalue ensuite directement cet état futur. Dans la fonction `choisir_meilleure_action`, cela se traduit par le calcul suivant : $\text{Scores} = \text{MLP}(s_{t+1})$

L'agent sélectionne ensuite l'action qui mène à l'état futur ayant obtenu le score d'évaluation le plus élevé : $\text{argmax}(\text{Scores})$.

Pour faciliter et accélérer la convergence (l'apprentissage) des réseaux de neurones, plusieurs techniques avancées ont été codées dans la fonction `choisir_meilleur_action`.

Il s'agit d'un filtre dynamique appliqué avant que le réseau de neurones ne fasse son évaluation.

- **Élagage directionnel** : Les joueurs sont contraints de jouer vers l'avant (l'équipe 1 ne peut pas générer de force $F_x < 0$ et l'équipe 2 de force $F_x > 0$).

- **Confinement spatial** : Si un joueur se trouve aux limites physiques du terrain (x le 0, y hauteur, etc.), les actions le poussant vers l'extérieur sont retirées de la liste des choix possibles.
- **Avantage théorique** : Grâce à ce masquage, le réseau ne gaspille pas de ressources à apprendre les lois physiques élémentaires ou les limites du terrain. L'espace de recherche est réduit, rendant l'apprentissage plus rapide et plus stable.

Au lieu de calculer toutes les combinaisons d'actions pour les N joueurs d'une équipe simultanément (ce qui provoquerait une explosion combinatoire exponentielle), l'équipe prend ses décisions de manière séquentielle et gloutonne :

1. Le joueur 1 évalue ses actions possibles en supposant que les autres ne font rien [zero]. Il verrouille sa meilleure action.
2. Le joueur 2 évalue ses actions en prenant en compte l'action que le joueur 1 vient de verrouiller.
3. Le processus se répète jusqu'au dernier joueur.

Cette approche permet de coordonner l'équipe tout en gardant un temps de calcul linéaire.

Dans la boucle principale du match(), le système utilise une politique d'exploration epsilon-greedy.

- Avec une probabilité epsilon, L'agent choisit une action totalement aléatoire parmi les actions valides. Cela permet au réseau de découvrir de nouvelles stratégies et d'éviter de rester bloqué dans un optimum local.
- Avec une probabilité $1 - \text{epsilon}$, L'agent exploite ses connaissances actuelles en utilisant la fonction `choisir_meilleure_action` pour maximiser la performance.

B - Partie physique

Pour la physique du rugby, j'ai adapté mon précédent code qui modélise le sport de manière précise. Les joueurs sont définis selon les caractéristiques suivantes :

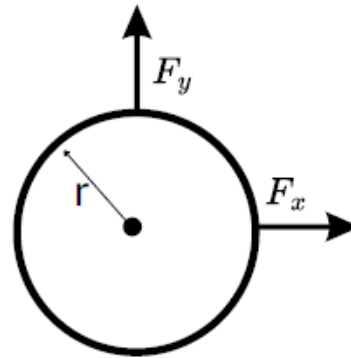
Paramètres :

$$x(t), y(t), v_x(t), v_y(t), a_x(t), a_y(t)$$

Rayon : r

Fatigue : $f(t)$

Masse : m

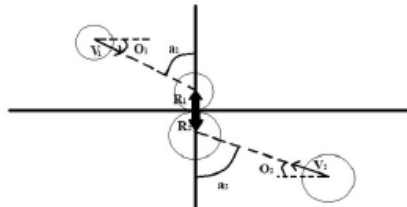


Déplacement :

À chaque pas de temps : le joueur choisit F_y et F_x pour se déplacer.

caractéristiques qui modélise 1 joueur

Les joueurs disposent d'une totale liberté de mouvement durant la simulation. Les plaquages sont modélisés par des contacts élastiques entre deux entités. Ils se déclenchent lors d'une collision entre les cercles de collision de deux joueurs, à condition que l'un d'eux possède le ballon. Chaque tentative de plaquage est soumise à une probabilité d'échec ; en cas de réussite, le défenseur accompagne le porteur du ballon au sol.



$$m_{tot} = m_1 + m_2$$

$$\theta = \arctan\left(\frac{m_1 v_1 \sin \theta_2}{m_1 v_1 \cos \theta_1 + m_2 v_2 \cos \theta_2}\right)$$

$$Vitesse = \frac{m_1 v_1 \cos \theta_1 + m_2 v_2 \cos \theta_2}{m_{tot} \cos \theta}$$

images de la modélisation du plaquage

Lorsqu'un plaquage est validé, une phase de ruck (mêlée ouverte) s'enclenche. Les joueurs à proximité se dirigent vers le regroupement pour tenter de récupérer le ballon. L'équipe qui remporte le ballon est déterminée par une équation calculant la somme de l'énergie cinétique grâce à la force fournie par chaque joueur engagé dans le ruck.

$$S_{equipe} = \sum_{i=0}^{n_j} m_i v_i^2$$

La fatigue est une variable dynamique prise en compte pour chaque agent. À chaque action effectuée, l'indice de fatigue du joueur augmente. Cet état impacte directement ses performances : la réussite des actions techniques devient plus aléatoire et sa vitesse de course maximale est réduite.

Probabilités	Fatigue
Plaquage	déplacement $\pm$ 0.01
Ruck	chaque action + 1
Interception	

différent impact de la fatigue

Résultats

Mon objectif principal était de voir émerger des stratégies de jeu cohérentes. Lors de la phase de développement, j'ai optimisé le code pour atteindre un temps d'entraînement maximal de 10 secondes par époque. Cela m'a permis d'entraîner mes réseaux de neurones sur un cycle de 1 000 époques. À l'issue de cet apprentissage, j'ai obtenu deux jeux de paramètres (poids synaptiques) définissant deux profils d'équipes distincts : l'un orienté vers l'attaque et l'autre vers la défense.

En testant ces deux modèles sur plusieurs matchs, j'ai constaté que la stratégie de progression offensive était presque systématiquement la même : les joueurs avancent en

s'échangeant plusieurs passes, cherchant à fixer les deux défenseurs sur un seul porteur de balle pour créer un décalage.

J'ai également remarqué que si j'inversais les rôles des deux équipes, les agents semblaient désorientés. Leurs décisions devenaient beaucoup moins efficaces, car l'entraînement n'avait pas été conçu pour gérer cette inversion de rôles (spécialisation excessive).

Piste d'évolutions & bilan personnel

Plusieurs axes d'amélioration ont émergé au cours de ce projet. Tout d'abord, un point essentiel serait d'augmenter le nombre de joueurs afin de simuler des situations de jeu plus complexes et variées. Cette évolution est directement liée à la puissance de calcul disponible : pour maintenir un temps d'apprentissage raisonnable tout en gérant davantage d'agents, l'utilisation de machines plus performantes — comme celles de la salle IA — serait un atout majeur.

Une autre piste de développement consisterait à entraîner plusieurs équipes aux profils variés, puis à croiser leurs apprentissages. Cette méthode permettrait d'obtenir une meilleure capacité d'adaptation des modèles, tout en optimisant le temps d'entraînement global.

Pour dresser un bilan de ce projet, j'ai initialement envisagé un périmètre trop vaste par rapport au temps imparti. J'ai donc dû faire preuve d'adaptabilité pour redéfinir des objectifs concrets et atteignables pour la soutenance finale. Ce travail m'a permis d'approfondir considérablement mes connaissances sur les principes du Reinforcement Learning (Apprentissage par Renforcement). Enfin, cette expérience a renforcé ma motivation à poursuivre le développement de ce projet à titre personnel.

Comment utiliser mon code

Tout d'abord, l'utilisation de ce programme nécessite un environnement d'exécution pour le langage Python. Si vous n'en possédez pas, je vous recommande d'utiliser Visual Studio Code (VS Code) ou la plateforme en ligne Google Colab.

La procédure se déroule en deux étapes :

1. Apprentissage : Il suffit d'exécuter le script `main.py` et de laisser le processus d'entraînement aller jusqu'à son terme.
2. Évaluation : Une fois les modèles entraînés, vous pouvez lancer le script `test_ia.py`. Celui-ci permet de charger les paramètres sauvegardés et de tester les performances de l'IA selon les configurations souhaitées.